

Lógica Fuzzy na Classificação de Anomalias em APIs REST

Fuzzing guiado por contrato + Sistema de Inferência Fuzzy (Mamdani)

Levi Gomes - Luiz Garcia - Natã Cezer Bordignon - Renan Balestrin Bez

Contexto e problema

APIs REST são base de microsserviços, apps móveis e integrações B2B. Para testar robustez, o fuzzer usa o contrato OpenAPI e envia entradas mutantes.

O desafio é o **oráculo de teste**: decidir se uma resposta inesperada é uma falha real ou um comportamento aceitável sob estresse.

Abordagens binárias tratam achados muito diferentes como iguais:

- 400 não documentado
- 200 aceitando payload inválido
- 500 por exceção interna

Oráculo booleano: falha / passa

Lógica Fuzzy: score contínuo [0, 1]

Pergunta e objetivo

Pergunta de pesquisa: um Sistema de Inferência Fuzzy (FIS) consegue classificar anomalias de APIs REST com mais granularidade que uma abordagem booleana?

O FIS combina três variáveis de entrada:

- `status_deviation` — desvio do código HTTP
- `response_time` — latência da resposta
- `payload_ratio` — tamanho relativo do payload

Gera um **score contínuo entre 0.0 e 1.0**, convertido em `LOW`, `MEDIUM` ou `HIGH`.

Score contínuo — não binário

Por que Lógica Fuzzy?

Lógica Booleana

Fronteiras rígidas

- resposta é "normal" **ou** "lenta"
- payload é "esperado" **ou** "grande"
- desvio é "presente" **ou** "ausente"

Lógica Fuzzy

Graus de pertinência ([0, 1])

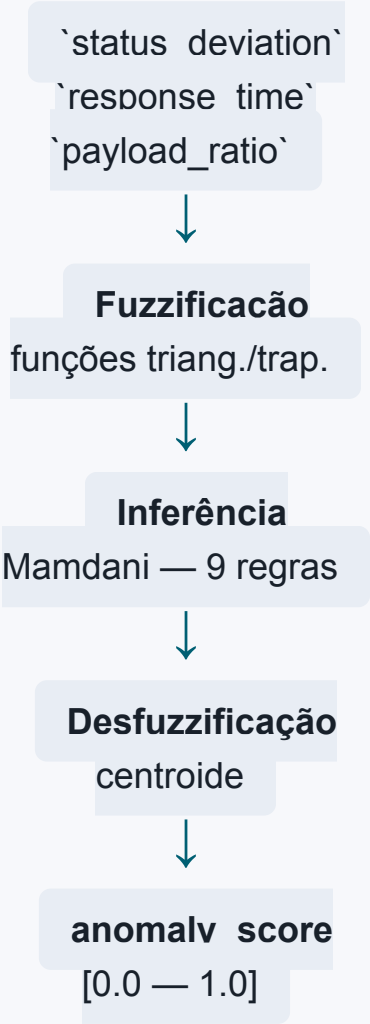
- resposta: 30% normal + 70% lenta
- payload: 60% esperado + 40% grande
- desvio: baixo, médio ou alto

As **funções de pertinência triangulares e trapezoidais** mapeiam cada valor observado para o intervalo ([0, 1]), capturando a incerteza natural de métricas de rede como latência e tamanho de resposta.

Variáveis e fluxo do FIS

Variável	Conjuntos
status_deviation	low, medium, high
response_time	normal, slow, timeout
payload_ratio	expected, large, empty
anomaly_score	low, medium, high

LOW [0, 0.33) · MEDIUM [0.33, 0.66) · HIGH [0.66, 1.0]



Regras de inferência

As regras traduzem **conhecimento heurístico** em decisões automatizadas. **9 regras** no total — interpretáveis e ajustáveis:

SE desvio é alto OU tempo é timeout	→ criticidade ALTA
SE desvio é alto	→ criticidade ALTA
SE desvio é médio E tempo é lento	→ criticidade MÉDIA
SE desvio é baixo E payload é grande	→ criticidade MÉDIA
SE desvio é baixo E tempo é normal E payload ok	→ criticidade BAIXA

Inferência Mamdani

Desfuzzificação por centroide

Regras interpretáveis

Metodologia: status e mutações

Desvio de status

Situação	Desvio
Status documentado	0.0
Sem referência clara	0.5
4xx não documentado	0.6
5xx ou sem resposta	1.0

Mutações (entrada do FIS)

Nível	Descrição
Level 0	Baseline válido
Level 1	1 violação por vez (tipo, vazio, overflow)
Level 2	2 violações simultâneas

O valor de `status_deviation` alimenta o FIS com um **grau de desvio** — quanto maior, maior o impacto nas regras de inferência.

Metodologia: auth, classificação e ambiente

O sistema testa endpoints protegidos com auto-auth e probes de autenticação.

Pipeline da análise:

Resposta bruta
→ classificador binário (tipo da anomalia)
→ FIS (score contínuo)
→ severidade (LOW/MEDIUM/HIGH)

Tipos: `SERVER_ERROR` , `UNEXPECTED_SUCCESS` ,
`UNEXPECTED_ERROR_STATUS` , `ERROR_CONTRADICTION` .

Item	Configuração
Linguagem	Python 3.13
Biblioteca	scikit-fuzzy
API alvo	crAPI
Endpoints	cerca de 60
Fuzzing	Level 1
Limite	até 10 mutações / endpoint

Resultado geral

207

anomalias identificadas

42

classificadas como **HIGH** (≥ 0.66)

20,3% dos achados priorizados como críticos

Severidade	Intervalo	Quantidade
HIGH	≥ 0.66	42
MEDIUM	$[0.33, 0.66)$	120
LOW	< 0.33	45

O score contínuo do FIS permitiu reduzir a prioridade imediata de 207 para 42 anomalias.

Tipos de anomalia encontrados

Classificação	Quantidade	Percentual
UNEXPECTED_ERROR_STATUS	103	49,8%
SERVER_ERROR	42	20,3%
UNEXPECTED_SUCCESS	28	13,5%
ERROR_CONTRADICTION	22	10,6%
EXPECTED_FAILURE	10	4,8%
SUCCESS_CONTRADICTION	2	1,0%

A maioria foi documentação incompleta de erros 400 ; os casos mais graves foram erros 500 e 503 .

Casos representativos

Caso	Exemplo	Score	Severidade
Erro de servidor	/verify-email-token retornou 500	0.84	HIGH
Erro não documentado	login retornou 400 fora do contrato	0.60	MEDIUM
Sucesso inesperado	/reset-password aceitou payload inválido	0.50	MEDIUM
Rejeição correta	erro 400 documentado	0.31	LOW

O FIS diferencia falhas graves, drift de contrato e casos de baixa prioridade.

Discussão

Pontos fortes

- priorização clara das falhas críticas
- **score contínuo** em vez de rótulo binário
- combinação de contrato, desempenho e payload
- regras interpretáveis e ajustáveis
- potencial para execução em CI/CD

Limitações

- muitos 400 ficaram em severidade média
- thresholds podem ser ajustados
- regras ainda são estáticas, sem feedback adaptativo

Conclusão e próximos passos

A Lógica Fuzzy mostrou-se adequada para tratar a **incerteza** nos testes de robustez de APIs REST, substituindo a decisão binária por um **score contínuo** baseado em graus de pertinência e regras interpretáveis.

Comparada a um oráculo booleano, a abordagem reduziu a prioridade imediata de **207 achados** para **42 anomalias HIGH**.

Próximos passos:

- ajustar funções de pertinência e thresholds
- adicionar feedback do desenvolvedor
- detectar stack traces no corpo da resposta
- integrar o score fuzzy a pipelines de CI/CD